All for one and none forall

Compiling polymorphic relations without monomorphization

Dmitri Volkov, Yafei Yang, Chung-chieh Shan

2026 August 24

miniKanren vs. semiringKanren

miniKanren

- evaluation via stream search (top-down)
- dyamically typed (s-expressions)
- relations are unweighted (usually)

semiringKanren

- evaluation via multidimensional array operations (bottom-up)
- statically typed (finite algebraic data types)
- relations are weighted

semiringKanren represents relations as finite multidimensional arrays.

Polymorphic relations need to represent arbitrarily many specifically-typed relations.

How to do both?

All for one and none forall

Represent all concrete instances of a polymorphic relation

using only a single concrete instance

to compile away type variables

Representing relations


```
(defrel (graph
  (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

from / to

	0	1	2	3
0	0	1	0	0
1	1	0	0	0
2	0	1	0	1
3	0	0	0	0

Represent graphs as relations or as adjacency matrices.
semiringKanren computes matrices for arbitrary relations!

Primitive relations

(== x y)

x \ y	0	1	2	3
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1

(=/= x y)

x \ y	0	1	2	3
0	0	1	1	1
1	1	0	1	1
2	1	1	0	1
3	1	1	1	0

`disj` is addition

```
(defrel (r0 (x : (Num 4)) (y : (Num 4)))
  (disj
    (== x 0)
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

conj is multiplication

```
(defrel (r1 (x : (Num 4)) (y : (Num 4)))
  (conj
    (== x 0)
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

`fresh` is summation

Vertices with incoming edges.


```
(defrel (incoming-edge-vertices (y : (Num 4)))
  (fresh (x : (Num 4))
    (graph x y)))
```

$$\sum_{x} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 & 1 \end{bmatrix}$$

Types

semiringKanren values inhabit finite algebraic data types.

			left		right		
`(left 0)`	:	`(Sum (Num 2) (Num 3))`	1	0	0	0	0
`(left 1)`	:	`(Sum (Num 2) (Num 3))`	0	1	0	0	0
`(right 0)`	:	`(Sum (Num 2) (Num 3))`	0	0	1	0	0
`(right 1)`	:	`(Sum (Num 2) (Num 3))`	0	0	0	1	0
`(right 2)`	:	`(Sum (Num 2) (Num 3))`	0	0	0	0	1

(semiringKanren also supports unit and product types)

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Computing relations

```
(defrel (graph (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 1))
    (conj (== from 2) (== to 3))))
```

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto$ (left 0)

$y \mapsto$ (right 0)

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto$ (right 2)

$y \mapsto$ (left 2)

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Sum swap

```
(defrel (sum-swap-3-3
  (x : (Sum (Num 3) (Num 3)))
  (y : (Sum (Num 3) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 3))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto$ `(left 0)`

$y \mapsto$ `(right 0)`

Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto$ (right 3)

$y \mapsto$ (left 3)

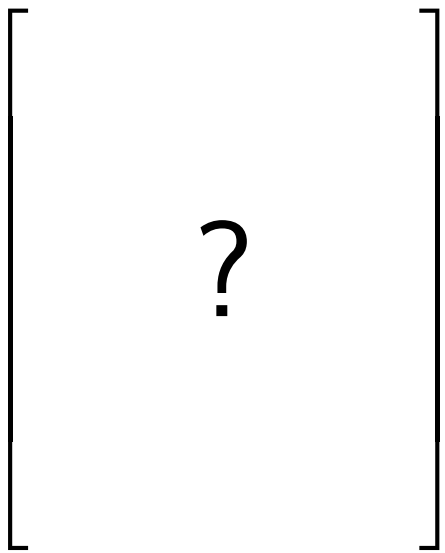
Larger sum swap

```
(defrel (sum-swap-3-4
  (x : (Sum (Num 3) (Num 4)))
  (y : (Sum (Num 4) (Num 3))))
  (disj
    (fresh (a : (Num 3))
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : (Num 4))
      (conj
        (== x (right b))
        (== y (left b))))))
```

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Polymorphic sum swap

```
(defrel (sum-swap
  (x : (Sum α β))
  (y : (Sum β α)))
  (disj
    (fresh (a : α)
      (conj
        (== x (left a))
        (== y (right a))))
    (fresh (b : β)
      (conj
        (== x (right b))
        (== y (left b))))))
```



All for one and none forall

Represent all concrete instances of a polymorphic relation

using only a single concrete instance

to compile away type variables

Expansion

semiringKanren computes a small array, and *expands* it.

Each expanded array entry adopts weight from a smaller array entry with the same *equality pattern*.

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$$x \mapsto (\texttt{left}\ \__\alpha)$$
$$y \mapsto (\texttt{left}\ \__\beta)$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$$x \mapsto (\texttt{right}\ \__\beta)$$
$$y \mapsto (\texttt{right}\ \__\alpha)$$

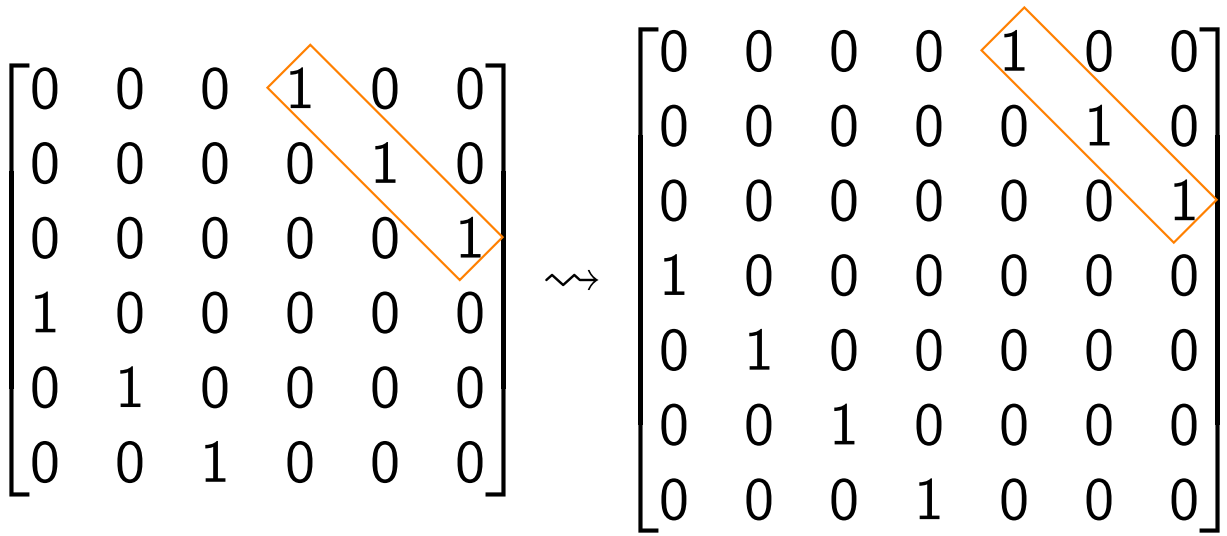
Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & \boxed{1} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Succeeds for:

$$\left.\begin{array}{r} x \mapsto (\texttt{left}\ \underline{0}) \\ y \mapsto (\texttt{right}\ \underline{0}) \end{array}\right\} \alpha\text{-typed parts equal}$$

Expanding sum swap



Succeeds for:

$$\left.\begin{array}{r} x \mapsto (\texttt{left}\ \__\alpha) \\ y \mapsto (\texttt{right}\ \__\alpha) \end{array}\right\} \alpha\text{-typed parts equal}$$

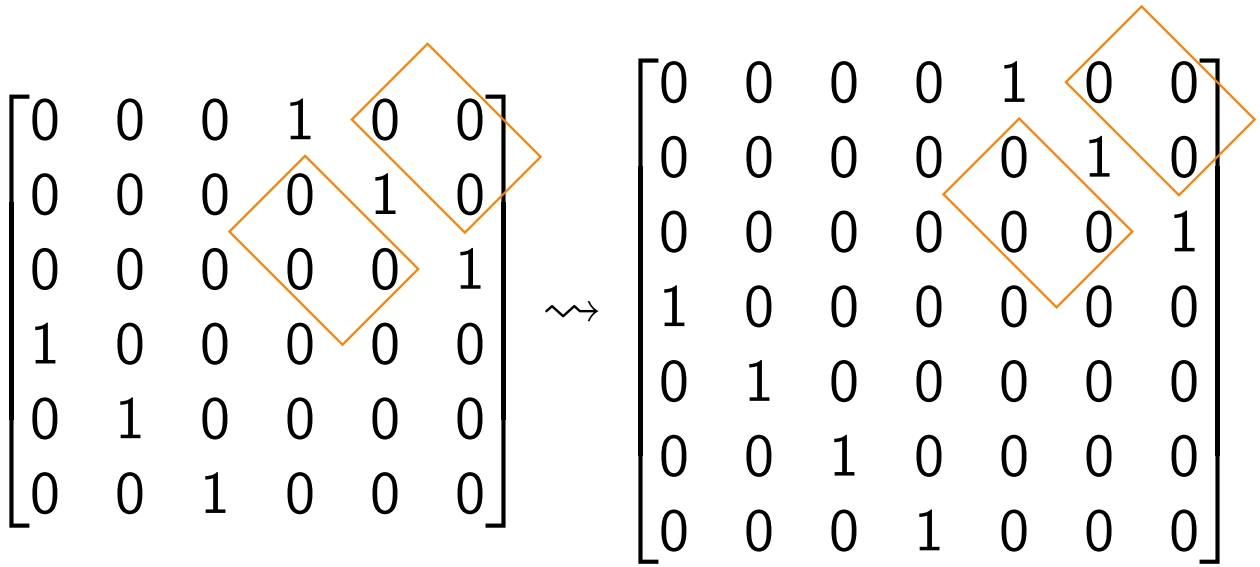
Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & \boxed{0} \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & \boxed{0} \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Fails for:

$$\left.\begin{aligned} x &\mapsto (\texttt{left}\ \underline{0}) \\ y &\mapsto (\texttt{right}\ \underline{2}) \end{aligned}\right\} \alpha\text{-typed parts disequal}$$

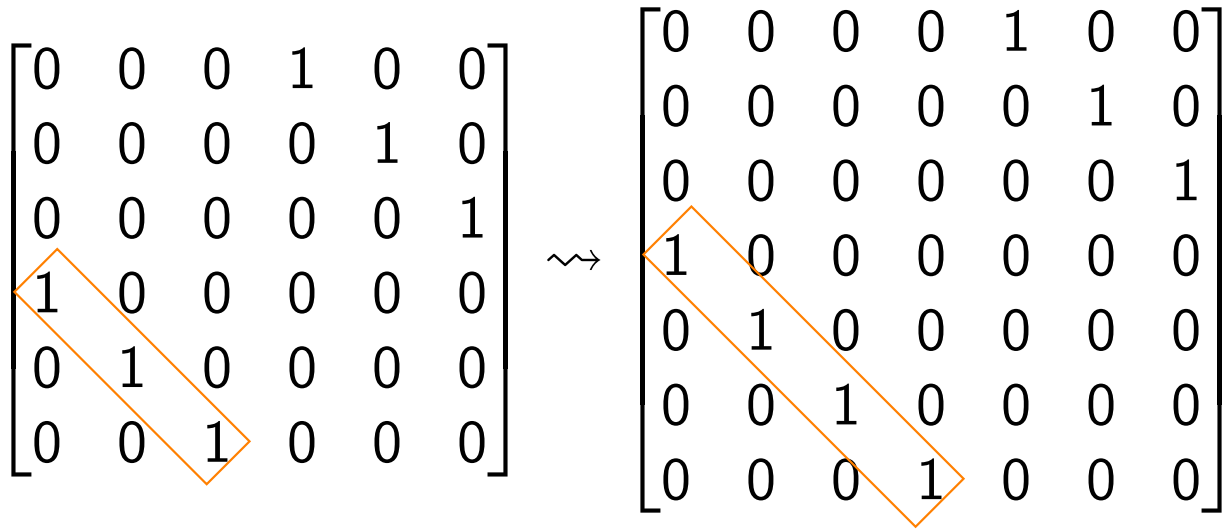
Expanding sum swap



Fails for:

$$\left.\begin{array}{r} x \mapsto (\texttt{left}\ \ \__\alpha) \\ y \mapsto (\texttt{right}\ \ \__\alpha) \end{array}\right\} \alpha\text{-typed parts disequal}$$

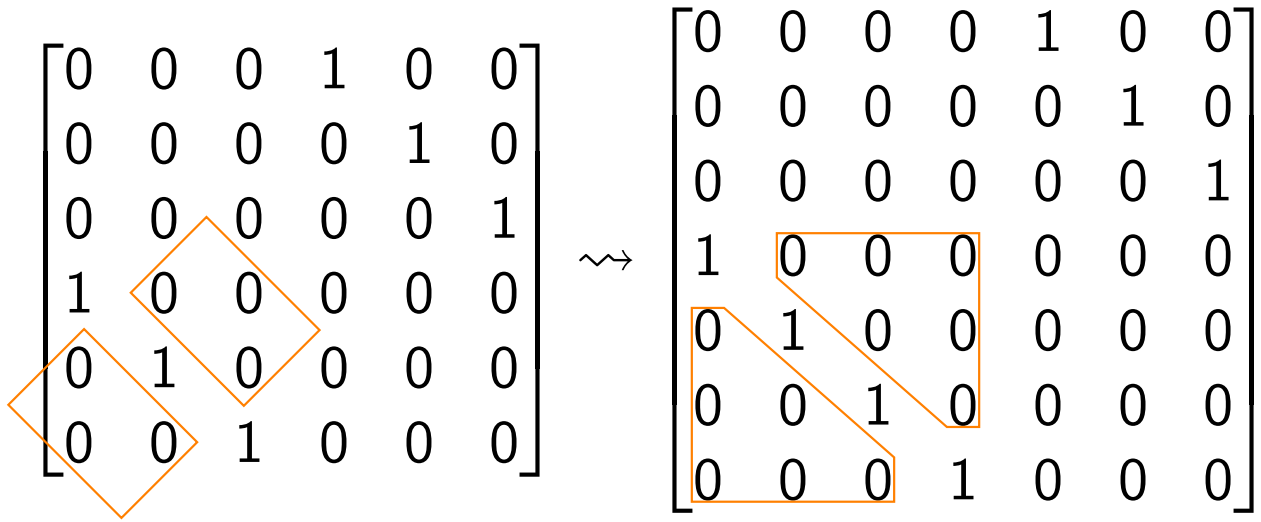
Expanding sum swap



Succeeds for:

$$\left.\begin{array}{r} x \mapsto (\texttt{right}\ \__\beta) \\ y \mapsto (\texttt{left}\ \__\beta) \end{array}\right\} \beta\text{-typed parts equal}$$

Expanding sum swap



Fails for:

$$\left.\begin{aligned} x &\mapsto (\texttt{right}\ \__\beta) \\ y &\mapsto (\texttt{left}\ \__\beta) \end{aligned}\right\} \beta\text{-typed parts disequal}$$

Expanding sum swap

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \rightsquigarrow \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Same success/failure results when:

- Non-variable typed parts are the same
- Variable-typed parts have the same (dis)equalities

Equality patterns

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

$x \mapsto (\texttt{right } \underline{2}_\beta)$

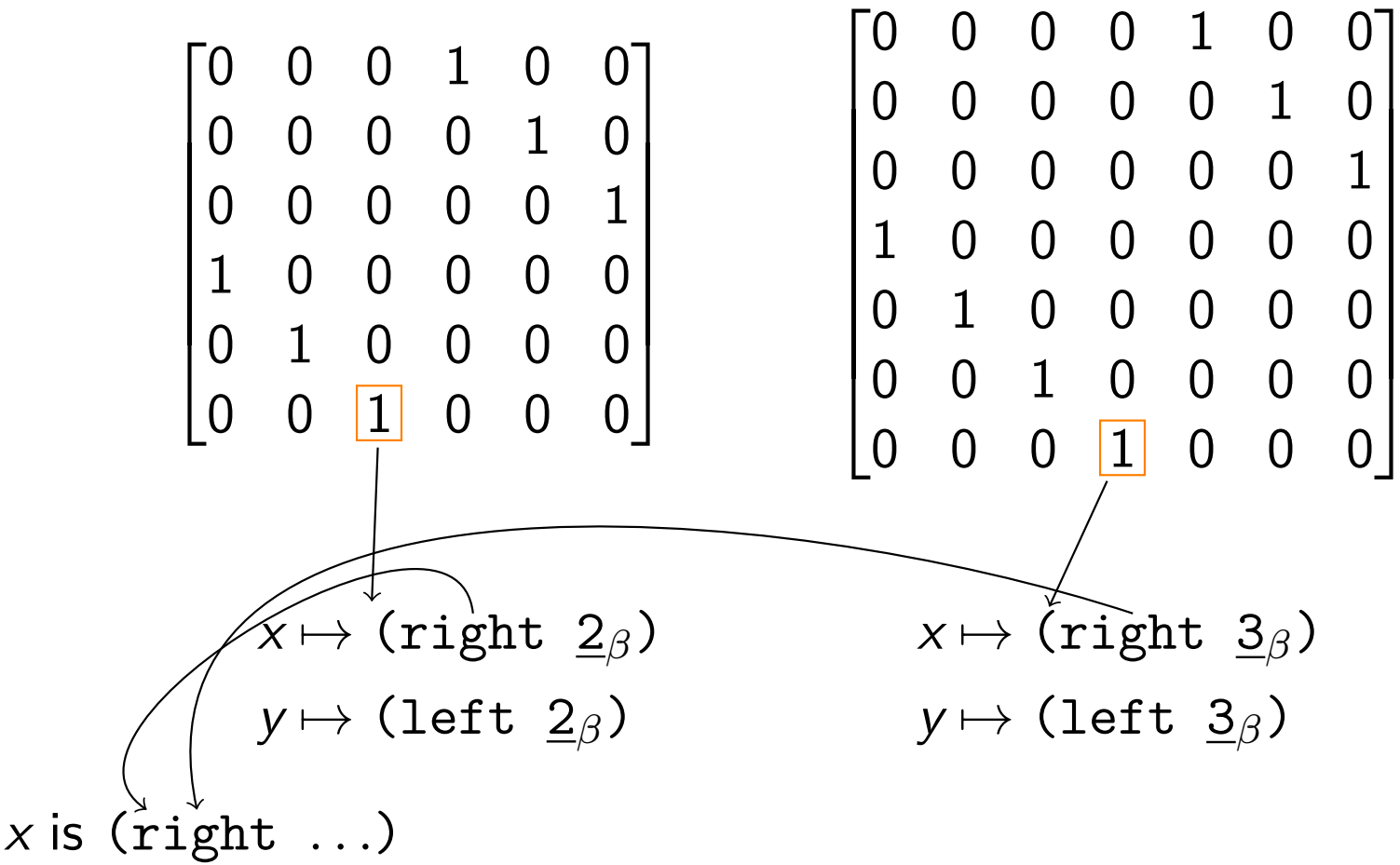
$y \mapsto (\texttt{left } \underline{2}_\beta)$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \boxed{1} & 0 & 0 & 0 \end{bmatrix}$$

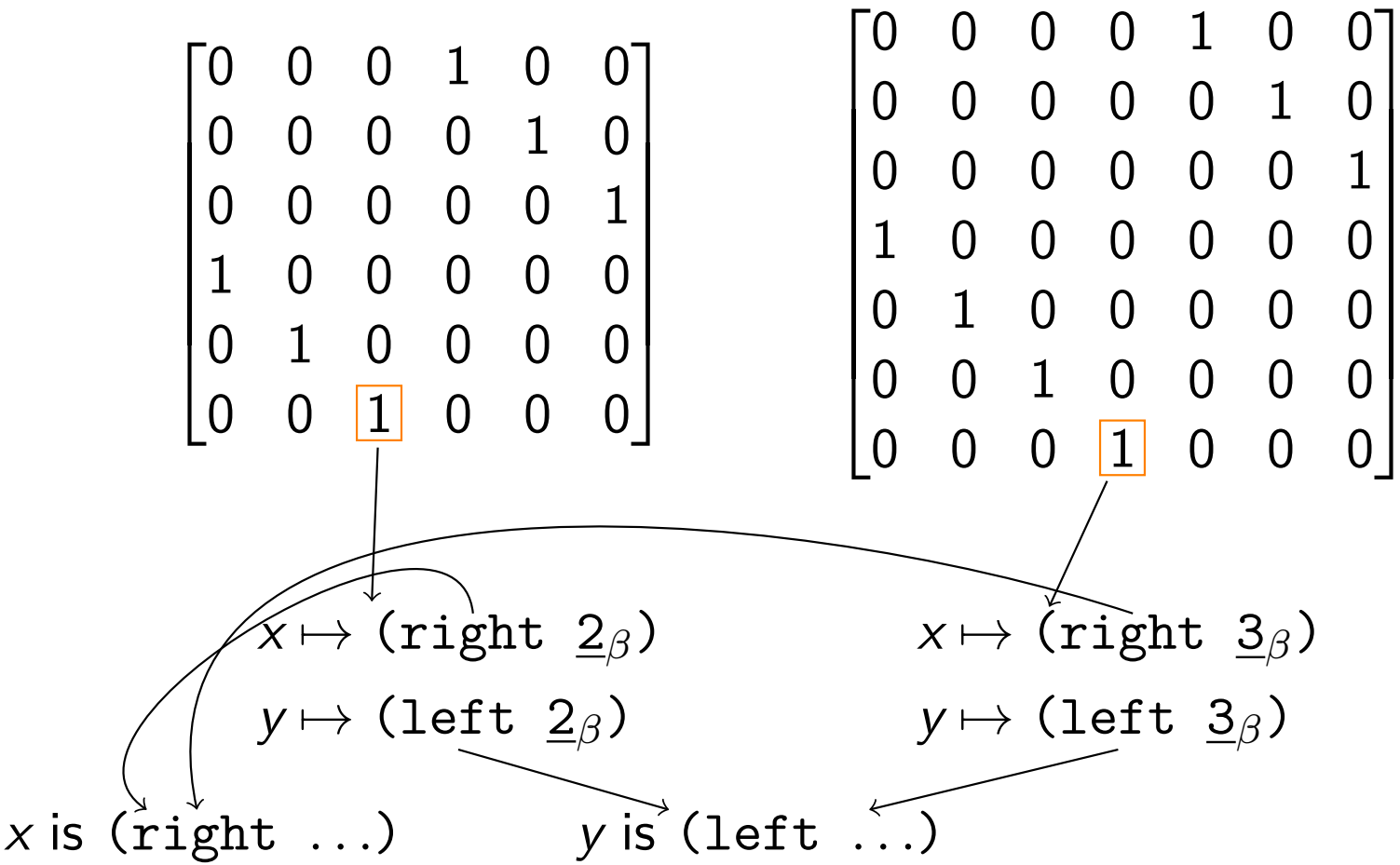
$x \mapsto (\texttt{right } \underline{3}_\beta)$

$y \mapsto (\texttt{left } \underline{3}_\beta)$

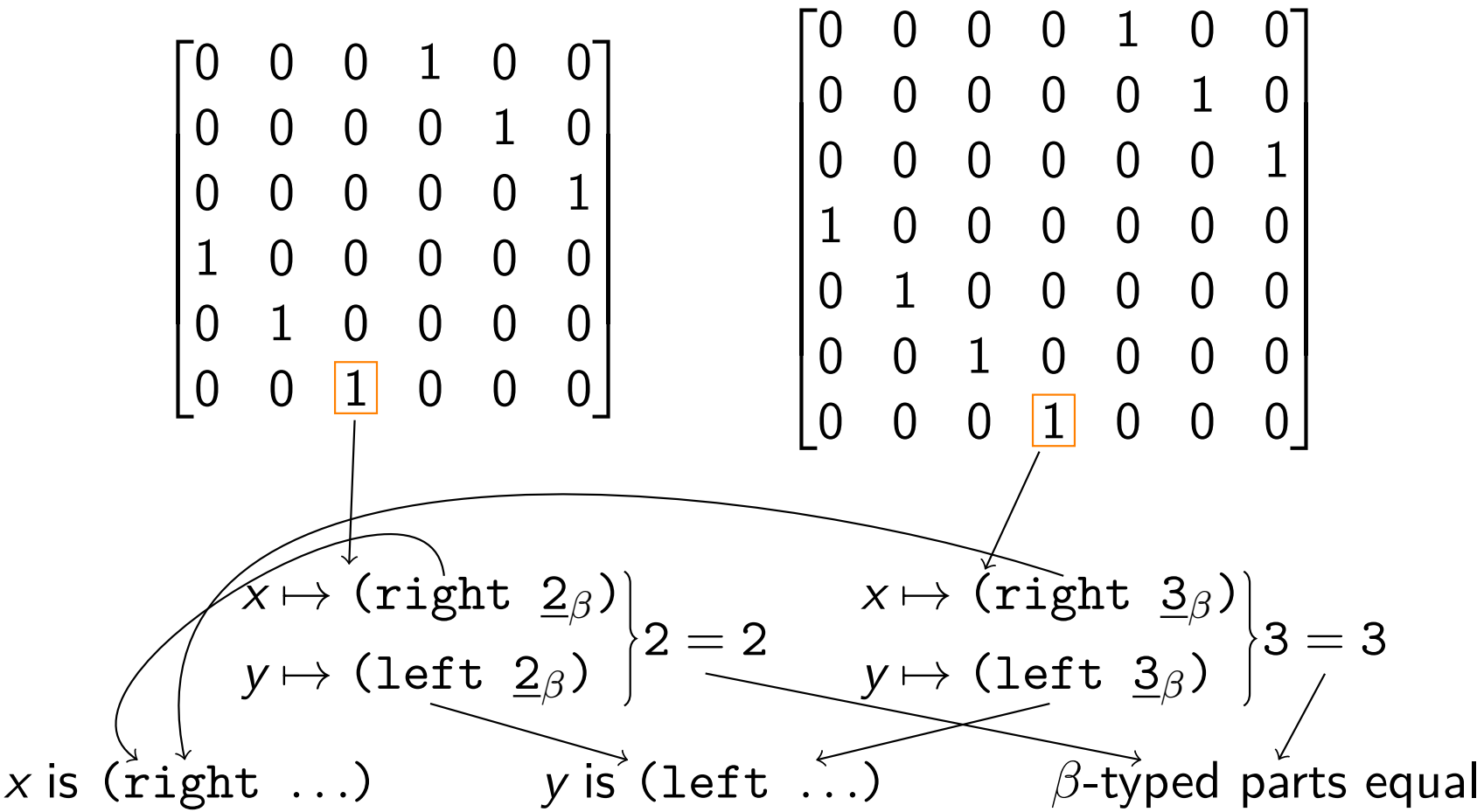
Equality patterns



Equality patterns



Equality patterns



Equality patterns

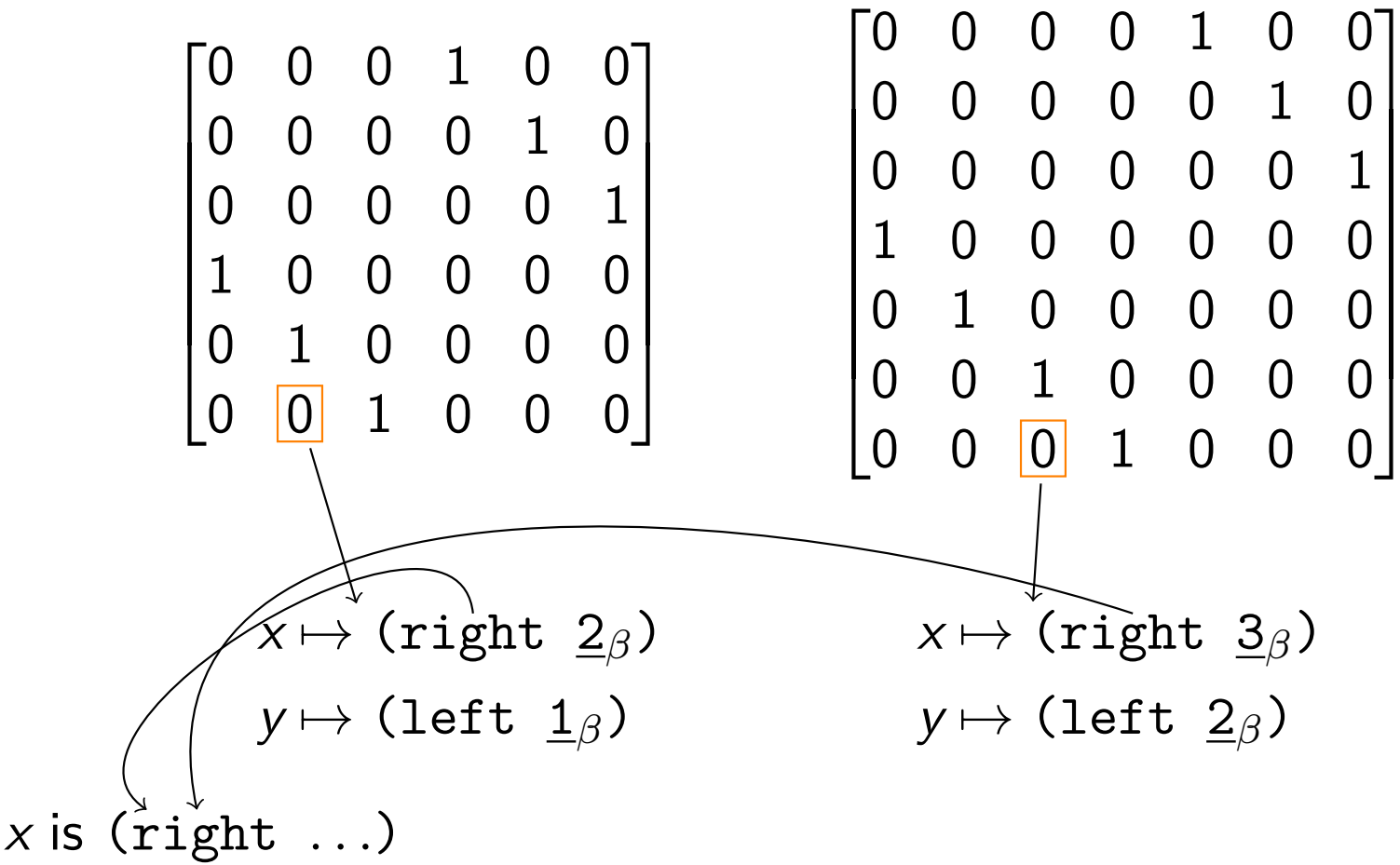
$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & \boxed{0} & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$x \mapsto (\texttt{right } \underline{2}_\beta)$$
$$y \mapsto (\texttt{left } \underline{1}_\beta)$$

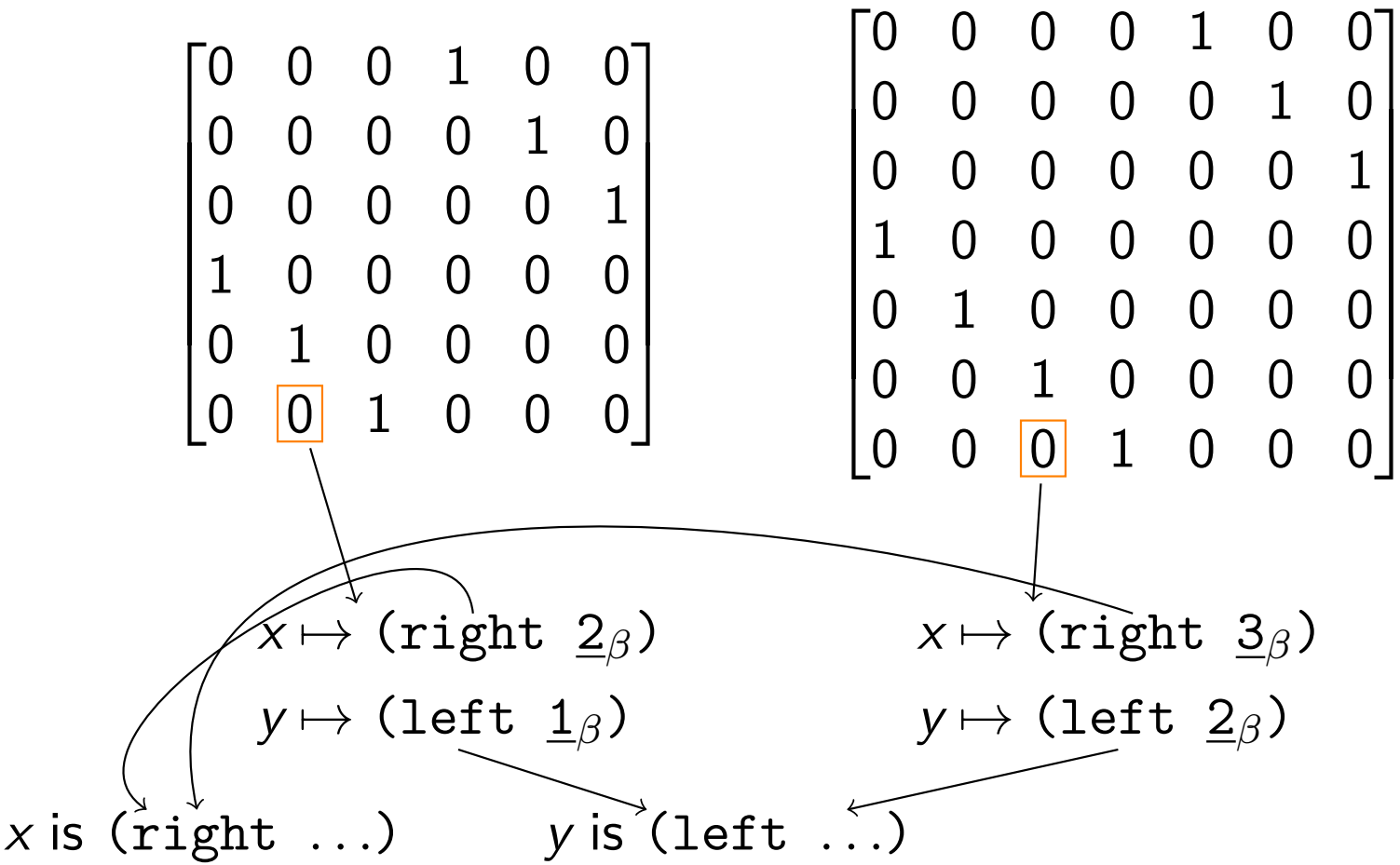
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \boxed{0} & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$x \mapsto (\texttt{right } \underline{3}_\beta)$$
$$y \mapsto (\texttt{left } \underline{2}_\beta)$$

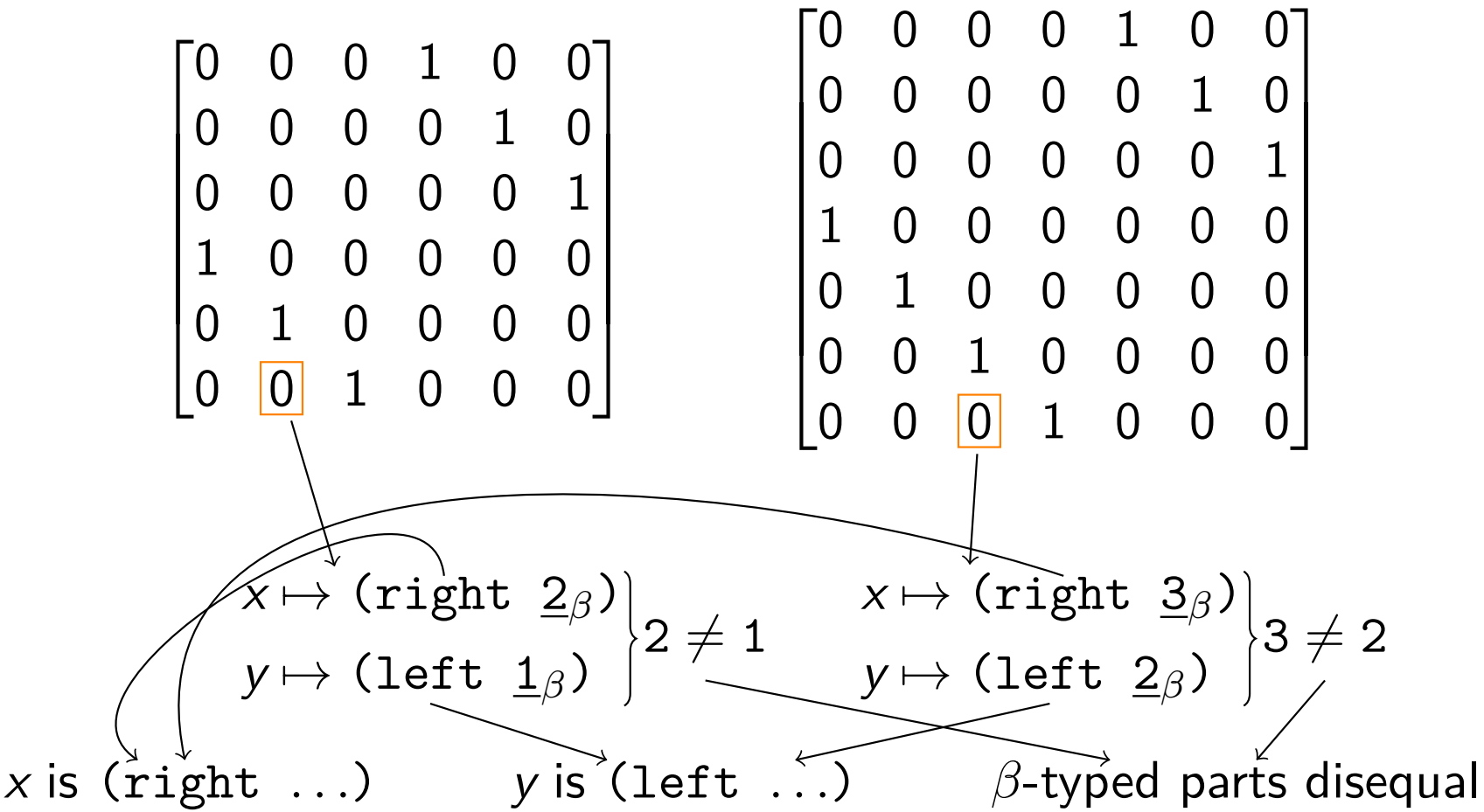
Equality patterns



Equality patterns



Equality patterns



Equality patterns

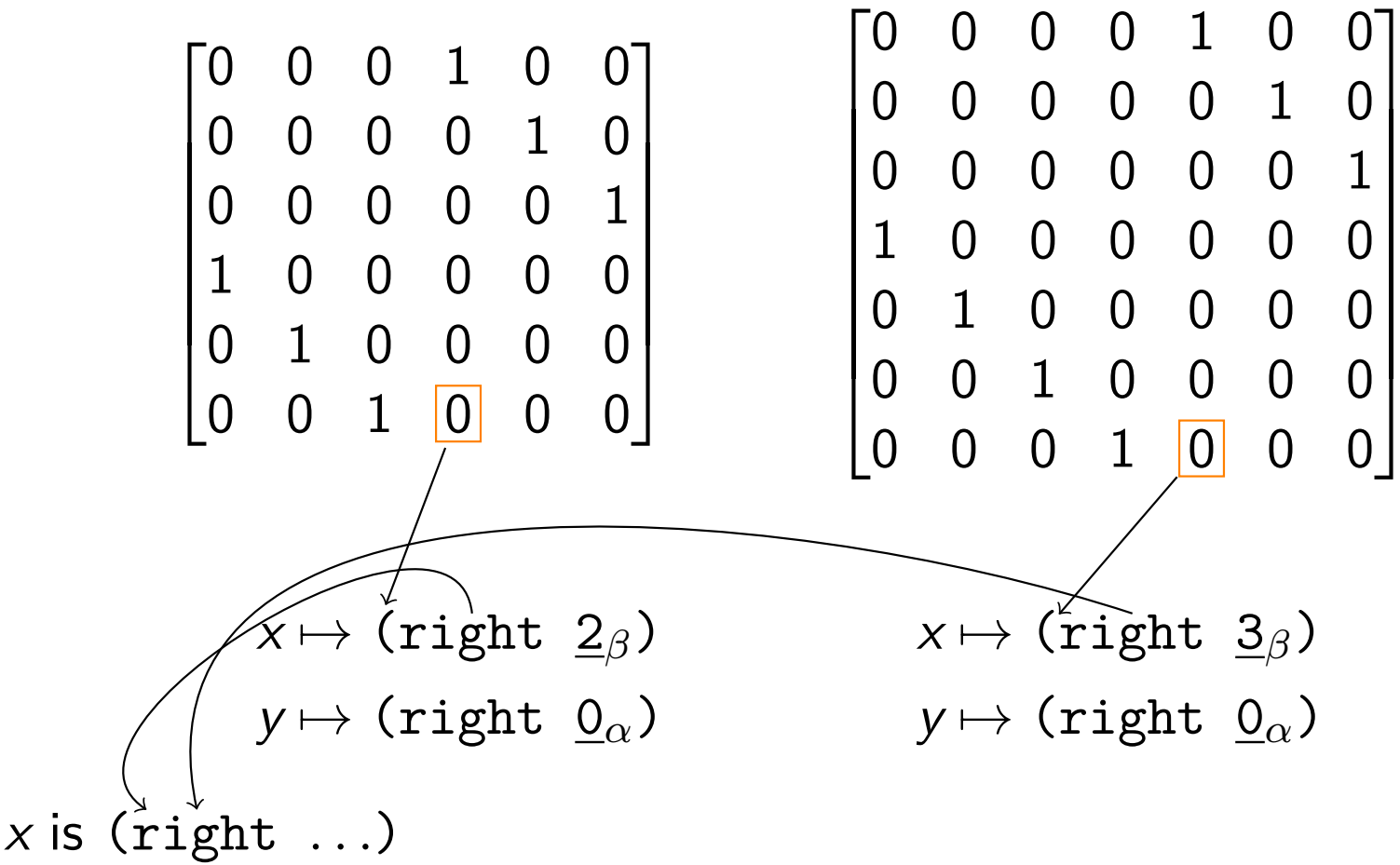
$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & \boxed{0} & 0 & 0 \end{bmatrix}$$

$$x \mapsto (\texttt{right } \underline{2}_\beta)$$
$$y \mapsto (\texttt{right } \underline{0}_\alpha)$$

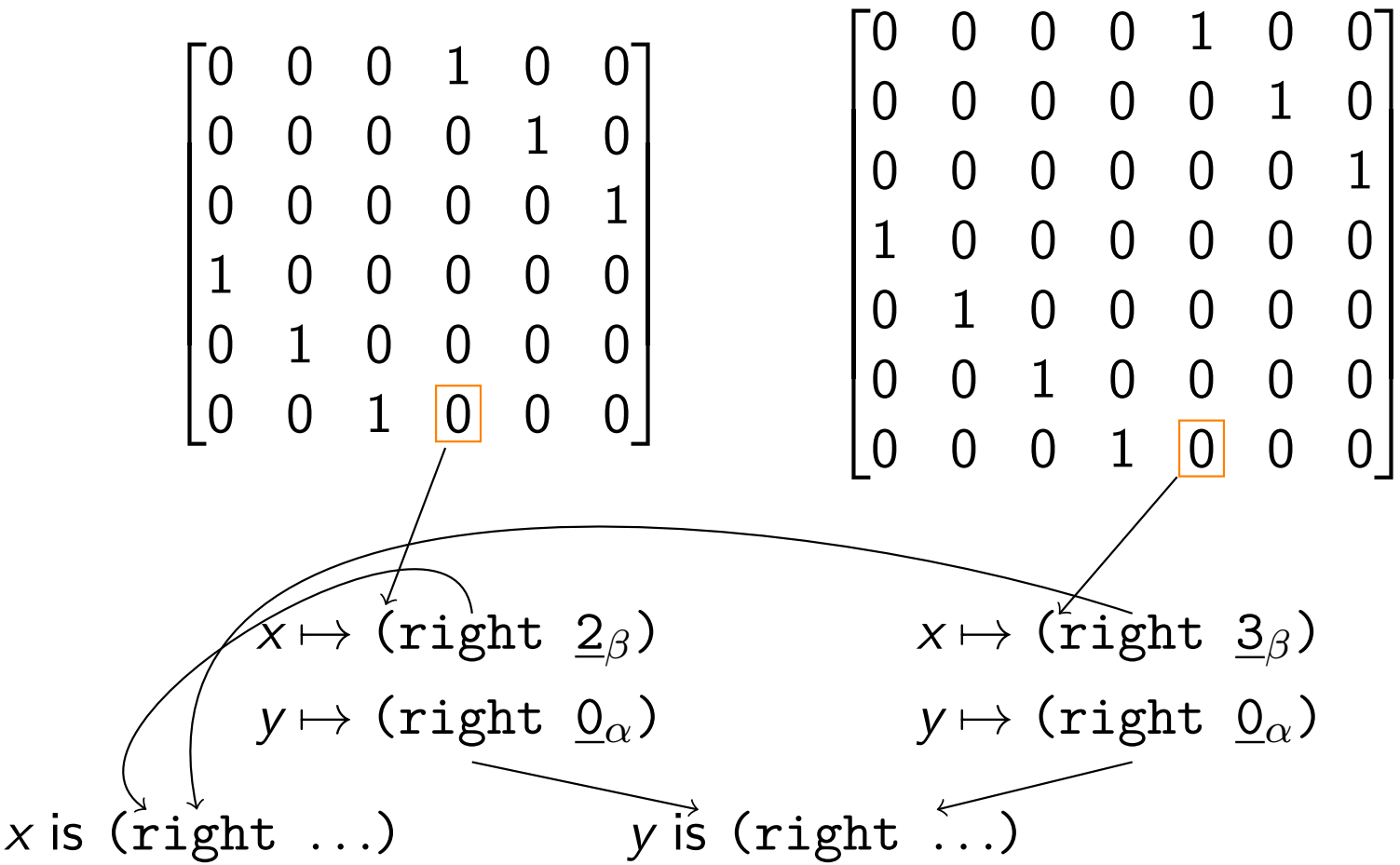
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & \boxed{0} & 0 & 0 \end{bmatrix}$$

$$x \mapsto (\texttt{right } \underline{3}_\beta)$$
$$y \mapsto (\texttt{right } \underline{0}_\alpha)$$

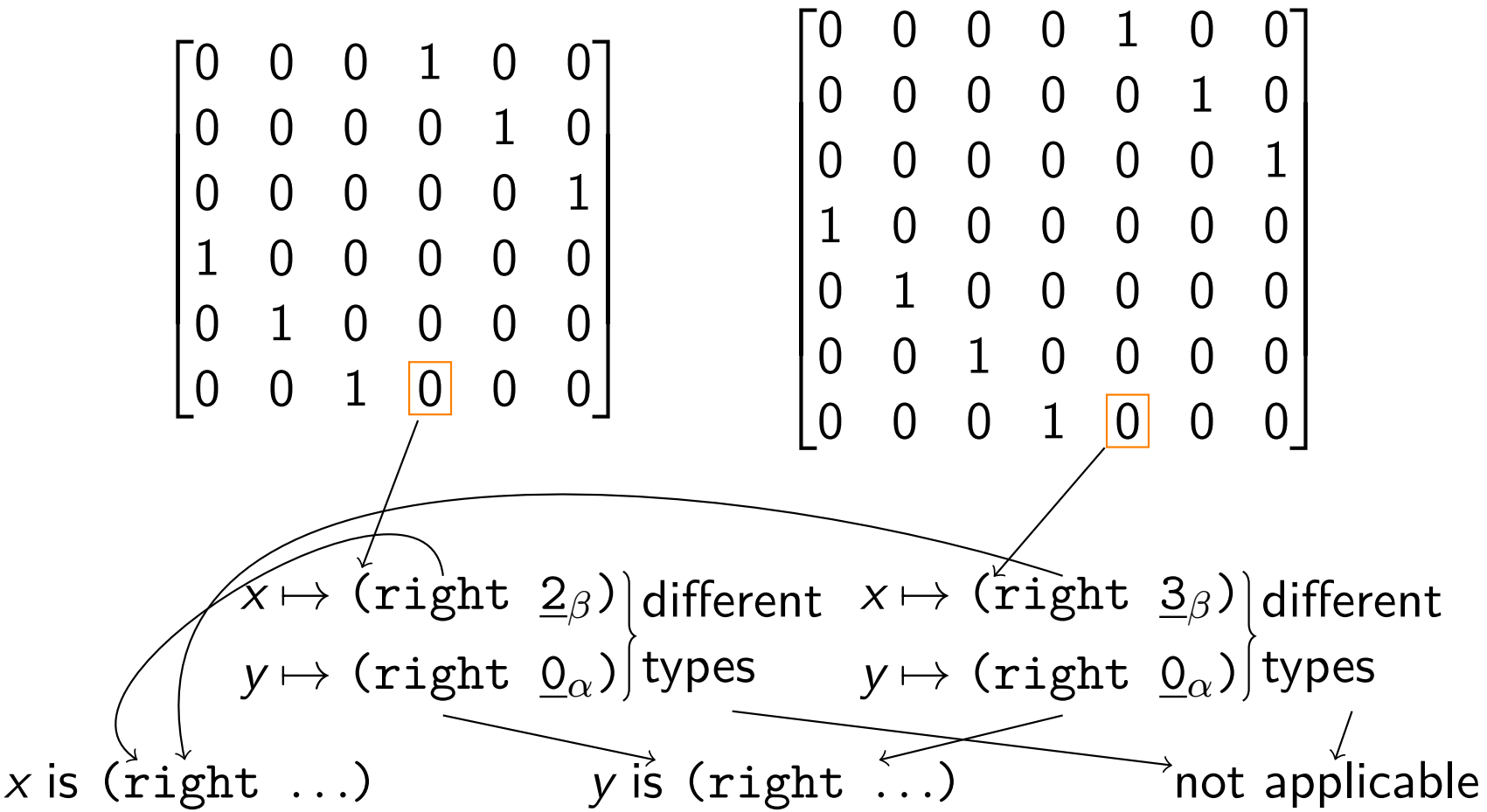
Equality patterns



Equality patterns



Equality patterns



We have:

- Computed non-polymorphic relations bottom-up

We have:

- Computed non-polymorphic relations bottom-up
- Compactly and concretely represented polymorphic relations
  - So can evaluate using the same methods as before

We have:

- Computed non-polymorphic relations bottom-up
- Compactly and concretely represented polymorphic relations
  - So can evaluate using the same methods as before

All done?

Has disequality

```
(defrel (has-disequality (y : α))
  (fresh (x : α)
    (=/= x y)))
```

Has disequality

```
(defrel (has-disequality (y : α))
  (fresh (x : α)
    (=/= x y)))
```

$\alpha = \texttt{(Num 2)}$

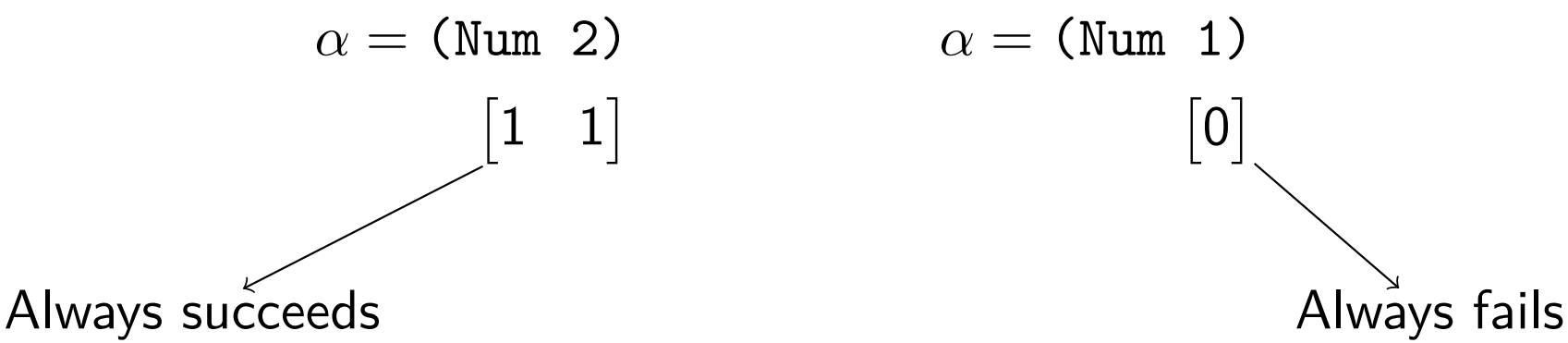
$$\begin{bmatrix} 1 & 1 \end{bmatrix}$$

$\alpha = \texttt{(Num 1)}$

$$\begin{bmatrix} 0 \end{bmatrix}$$

Has disequality

```
(defrel (has-disequality (y : α))
  (fresh (x : α)
    (=/= x y)))
```



Has disequality

```
(defrel (has-disequality (y : α))
  (fresh (x : α)
    (=/= x y)))
```

α = (Num 2)

$$\begin{bmatrix} 1 & 1 \end{bmatrix}$$

Always succeeds

α = (Num 1)

$$\begin{bmatrix} 0 \end{bmatrix}$$

Always fails

Expansion does not work here.

Has disequality

```
(defrel (has-disequality (y : α))
  (fresh (x : α)
    (=/= x y)))
```

$\alpha = \texttt{(Num 2)}$

$$\begin{bmatrix} 1 & 1 \end{bmatrix}$$

Always succeeds

$\alpha = \texttt{(Num 1)}$

$$\begin{bmatrix} 0 \end{bmatrix}$$

Always fails

Expansion does not work here.

Need to handle small-sized types separately (not "all for one", but almost).

Compilation

2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
   - Large-enough calls all use the same relation
   - Small calls use the correctly-sized monomorphic relation

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
   - Large-enough calls all use the same relation
   - Small calls use the correctly-sized monomorphic relation

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
   - Large-enough calls all use the same relation
   - Small calls use the correctly-sized monomorphic relation
3. Expand large-enough monomorphic relation calls back to the right types

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
   - Large-enough calls all use the same relation
   - Small calls use the correctly-sized monomorphic relation
3. Expand large-enough monomorphic relation calls back to the right types

All done by generating vanilla semiringKanren code

Compilation

1. Calculate type sizes
2. Replace polymorphic relation calls with concrete (monomorphic) relation calls
   - Large-enough calls all use the same relation
   - Small calls use the correctly-sized monomorphic relation
3. Expand large-enough monomorphic relation calls back to the right types

All done by generating vanilla semiringKanren code

More details in the paper!

Conclusion

Represent the (dis)equality structure of relations compactly, and expand as needed.

Conclusion

Represent the (dis)equality structure of relations compactly, and expand as needed.

Works because == and =/= are the only primitive relations.

Conclusion

Represent the (dis)equality structure of relations compactly, and expand as needed.

Works because == and =/= are the only primitive relations.

Does this generalize?

- To more than 2 arguments/array dimensions? (hint: yes)
- To recursive relations? (hint: yes)
- To more than == and =/=?
- To recursive data types?
- To non-relational languages?